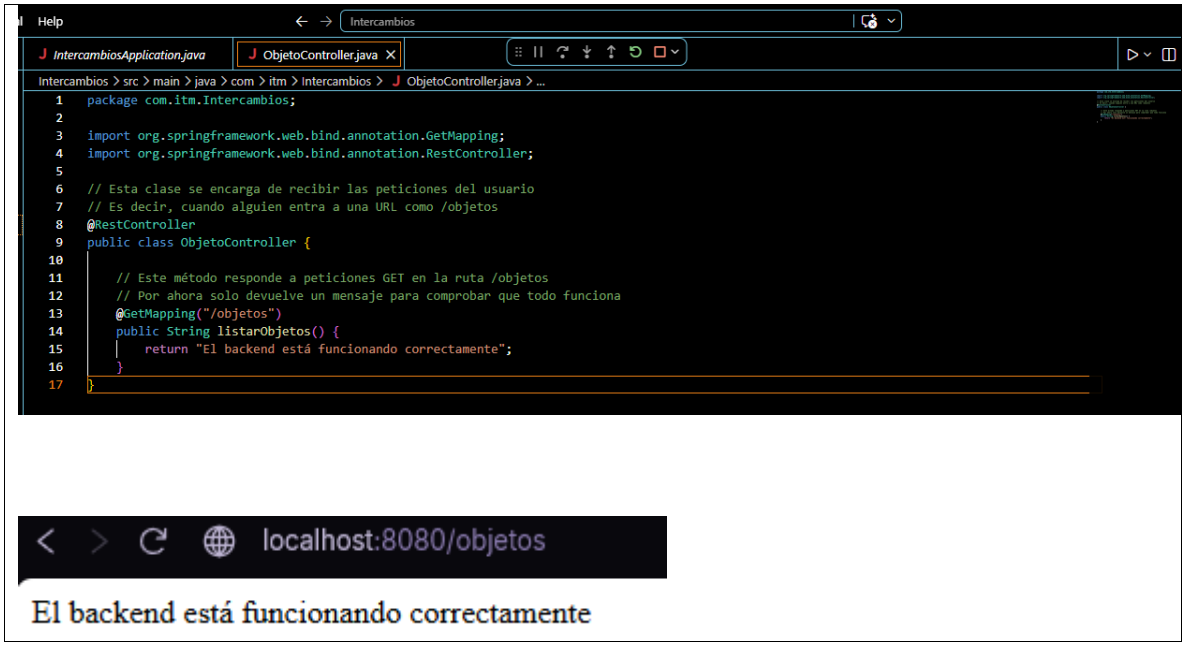
ENTREGABLE 2

ITM

8 DE ABRIL DE 2026

DAVID ALEJANDRO APOLINAR TRIANA/TOMAS GONZALES GARCIA



En esta parte del proyecto se creó una clase llamada ***ObjetoController***, la cual cumple la función de manejar las peticiones que hacen los usuarios al sistema.

Se utilizó la anotación ***@RestController***, que permite que la clase funcione como un controlador dentro del backend, es decir, que pueda recibir solicitudes desde el navegador o desde otras aplicaciones.

Dentro de esta clase se implementó un método llamado ***listarObjetos(),*** el cual está asociado a la ruta ***/objetos*** mediante la anotación ***@GetMapping***. Esto significa que cuando un usuario accede a la URL ***http://localhost:8080/objetos***, el sistema ejecuta este método.

Por el momento, este método retorna un mensaje de texto simple, el cual se utiliza para verificar que el servidor está funcionando correctamente y que el endpoint responde de manera adecuada.

Esta implementación inicial permite comprobar que el backend está correctamente configurado y listo para continuar con el desarrollo de las demás operaciones del sistema.

```java
package com.itm.Intercambios;

// Esta clase representa un "Objeto" dentro de nuestra aplicación
// Es decir, define los datos que cada objeto tendrá
public class Objeto {

    // Identificador único del objeto
    private int id;

    // Nombre del objeto
    private String nombre;

    // Descripción del objeto
    private String descripcion;

    // Constructor vacío (necesario para ciertas operaciones en Spring)
    public Objeto() {}

    // Constructor con todos los campos
    public Objeto(int id, String nombre, String descripcion) {
        this.id = id;
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    // Getters y setters permiten leer y modificar los campos

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    // Método para mostrar el objeto como texto
    @Override
    public String toString() {
        return "Objeto [id=" + id + ", nombre=" + nombre + ", descripcion=" + descripcion + "]";
    }
}
```

En esta parte del desarrollo se creó la clase **Objeto**, la cual representa la estructura de los datos que se manejarán en el sistema. Esta clase define las características que tendrá cada objeto dentro de la aplicación, como su identificador (**id**), su nombre (**nombre**) y su descripción (**descripcion**).
Se implementó un constructor vacío, necesario para el correcto funcionamiento de Spring, y un constructor con parámetros que permite crear objetos con todos sus atributos definidos desde el inicio.

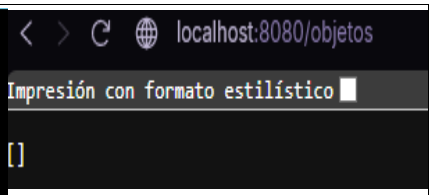
Además, se incluyeron los métodos **getters** y **setters**, los cuales permiten acceder y modificar los valores de cada atributo de manera controlada. Esto es importante para mantener una correcta manipulación de los datos dentro del sistema.
Por último, se implementó el método **toString()**, el cual permite representar el objeto en forma de texto, facilitando su visualización durante pruebas o procesos de depuración.
Esta clase será utilizada posteriormente para interactuar con la base de datos y para enviar información a través de los servicios web.

```java
package com.itm.Intercambios;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

// Esta clase maneja las peticiones del usuario
@RestController
public class ObjetoController {

    // Este método ahora devuelve una lista de objetos (no solo texto)
    @GetMapping("/objetos")
    public List<Objeto> listarObjetos() {

        // Creamos una lista de prueba
        List<Objeto> lista = new ArrayList<>();

        lista.add(new Objeto(id: 1, nombre: "Libro de matemáticas", descripcion: "Libro usado de cálculo"));
        lista.add(new Objeto(id: 2, nombre: "Calculadora", descripcion: "Calculadora científica"));

        return lista;
    }
}
```

```json
[
  {
    "id": 1,
    "nombre": "Libro de matemáticas",
    "descripcion": "Libro usado de cálculo"
  },
  {
    "id": 2,
    "nombre": "Calculadora",
    "descripcion": "Calculadora científica"
  }
]
```

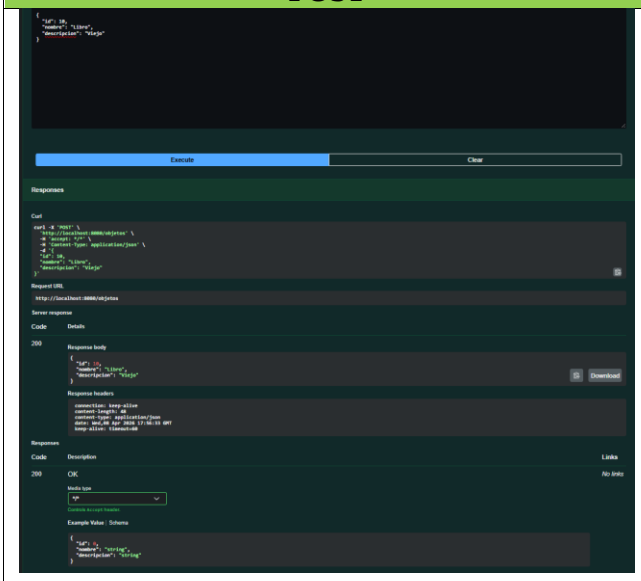
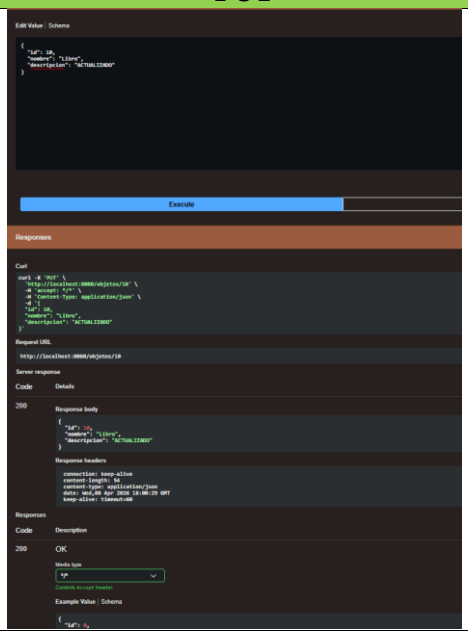
En esta etapa se modificó el controlador para que el endpoint **/objetos** devuelva una lista de objetos en lugar de un mensaje de texto.
Para esto, se creó una lista de tipo **List<Objeto>** dentro del método **listarObjetos()**, en la cual se agregaron objetos de prueba con información como el identificador, nombre y descripción.
El método está anotado con **@GetMapping("/objetos")**, lo que permite que al acceder a la URL correspondiente, el sistema ejecute este método y retorne los datos en formato JSON automáticamente.
Esto permite simular el funcionamiento real del sistema, donde los objetos representarían los artículos disponibles para intercambio entre estudiantes.
Esta implementación es importante, ya que demuestra cómo el backend puede enviar información estructurada que posteriormente será utilizada por otras partes del sistema, como el frontend o aplicaciones externas.

```java
package com.itm.Intercambios;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

// Esta clase maneja las peticiones del usuario
@RestController
public class ObjetoController {

    // Lista que guarda los objetos (simulación de base de datos)
    private List<Objeto> lista = new ArrayList<>();

    // GET → ver objetos
    @GetMapping("/objetos")
    public List<Objeto> listarObjetos() {
        return lista;
    }

    // POST → agregar un nuevo objeto
    @PostMapping("/objetos")
    public Objeto agregarObjeto(@RequestBody Objeto objeto) {
        lista.add(objeto);
        return objeto;
    }
}
```



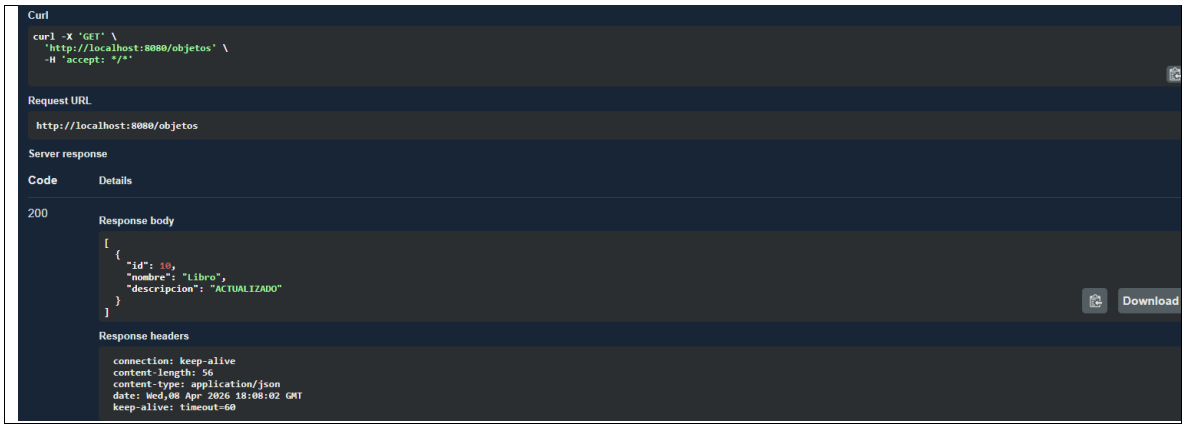
Durante el desarrollo del proyecto, el controlador **ObjetoController** fue cambiando poco a poco para hacerlo más funcional y más parecido a una aplicación real.
Al inicio, el endpoint GET solo devolvía un mensaje de texto para comprobar que el servidor estaba funcionando correctamente. Luego, se modificó para que devolviera una lista de objetos de prueba, lo que permitió ver cómo se envían los datos en formato JSON.
Después, se hizo un cambio importante: en lugar de usar datos fijos, se creó una lista en memoria que permite guardar los objetos que se van creando mientras el programa está en ejecución.
También se agregó el método POST usando **@PostMapping**, el cual permite que el usuario envíe información y se agreguen nuevos objetos a la lista. Esto simula cómo funcionaría un sistema real donde los usuarios pueden publicar artículos.
En general, estos cambios muestran cómo el proyecto fue pasando de una prueba básica a algo más interactivo, donde ya se pueden consultar y agregar datos.

DOCUMENTACION DEL PUT:

POST	GET	PUT
		

(Para mejor lectura hacer zoom)

Y ahora verificamos el cambio.



En esta parte del proyecto se implementaron los métodos principales de un servicio web tipo REST, específicamente GET, POST y PUT, los cuales permiten consultar, crear y modificar información dentro del sistema.
Inicialmente, se utilizó el método GET para verificar que el servidor funcionara correctamente y posteriormente para consultar la lista de objetos registrados. Este método devuelve los datos en formato JSON, lo que permite visualizar fácilmente la información almacenada.
Luego, se implementó el método POST, el cual permite agregar nuevos objetos al sistema. Para esto, se envían los datos en formato JSON, los cuales son recibidos por el backend y almacenados en una lista en memoria, simulando el comportamiento de una base de datos.
Finalmente, se implementó el método PUT, que permite actualizar la información de un objeto existente mediante su identificador (id). Durante las pruebas, se logró modificar correctamente los datos de un objeto, lo cual se comprobó realizando consultas posteriores con GET.
En conjunto, estos métodos permiten que el sistema sea interactivo, ya que los usuarios pueden consultar, registrar y modificar información, acercándose al funcionamiento de una aplicación real.

DELETE:

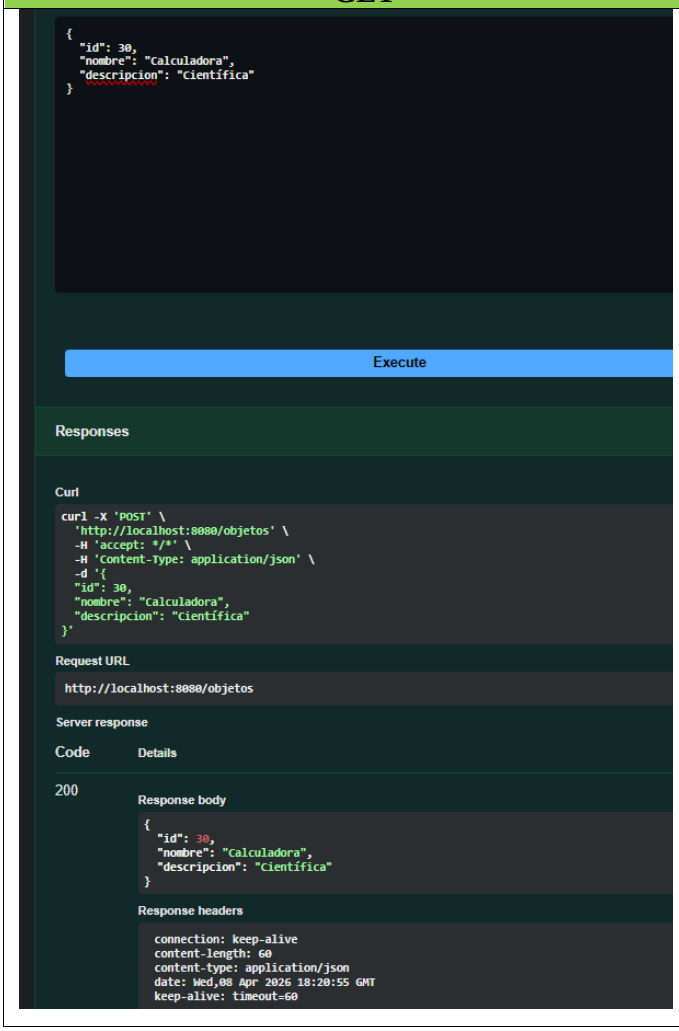
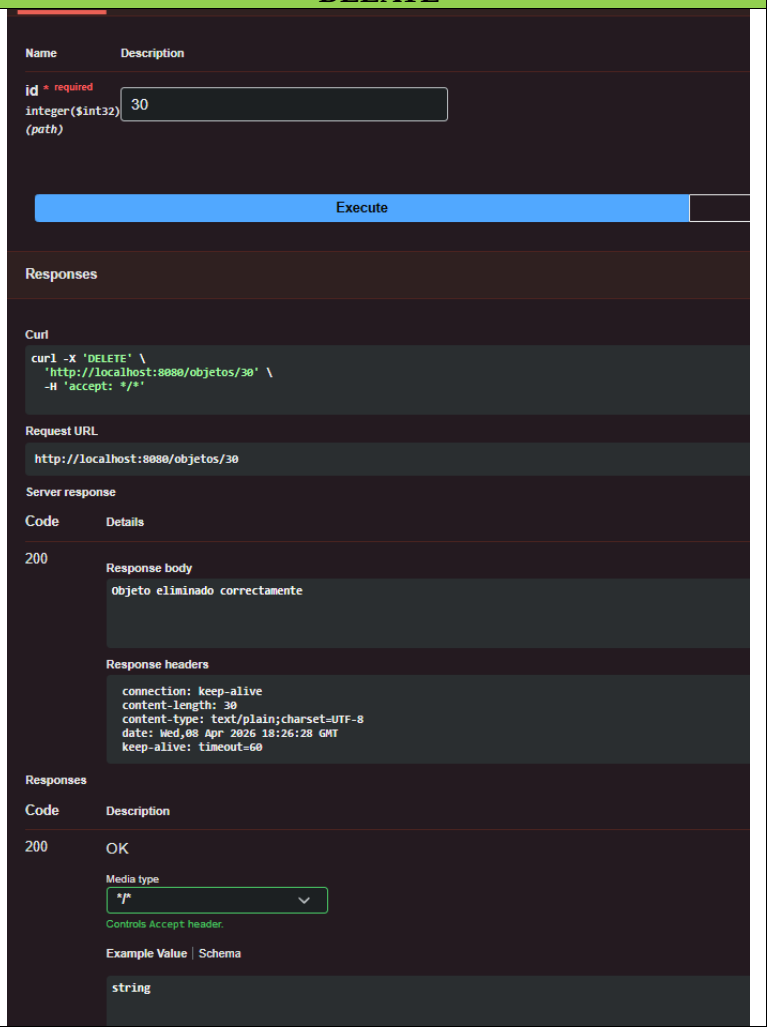
```java
package com.itm.Intercambios;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

// Esta clase maneja las peticiones del usuario
@RestController
public class ObjetoController {

    // Lista que guarda los objetos (simulación de base de datos)
    private List<Objeto> lista = new ArrayList<>();

    // GET → ver todos los objetos
    @GetMapping("/objetos")
    public List<Objeto> listarObjetos() {
        return lista;
    }

    // POST → agregar un nuevo objeto
    @PostMapping("/objetos")
    public Objeto agregarObjeto(@RequestBody Objeto objeto) {
        lista.add(objeto);
        return objeto;
    }

    // PUT → actualizar un objeto existente
    @PutMapping("/objetos/{id}")
    public Objeto actualizarObjeto(@PathVariable int id, @RequestBody Objeto objetoActualizado) {

        for (Objeto obj : lista) {
            if (obj.getId() == id) {
                obj.setNombre(objetoActualizado.getNombre());
                obj.setDescripcion(objetoActualizado.getDescripcion());
                return obj;
            }
        }

        return null; // si no se encuentra el objeto
    }

    // DELETE → eliminar un objeto por id
    @DeleteMapping("/objetos/{id}")
    public String eliminarObjeto(@PathVariable int id) {
```

```java
    // DELETE → eliminar un objeto por id
    @DeleteMapping("/objetos/{id}")
    public String eliminarObjeto(@PathVariable int id) {

        for (Objeto obj : lista) {
            if (obj.getId() == id) {
                lista.remove(obj);
                return "Objeto eliminado correctamente";
            }
        }

        return "Objeto no encontrado";
    }
}
```

En este fragmento de código se implementa el método **DELETE**, el cual permite eliminar un objeto de la lista utilizando su identificador (id).

El método recibe el id desde la URL mediante la anotación **@PathVariable**. Luego, recorre la lista de objetos buscando uno que coincida con ese id. Si lo encuentra, lo elimina de la lista y devuelve un mensaje confirmando la eliminación. En caso de no encontrar el objeto, se retorna un mensaje indicando que no existe.

Esta implementación permite gestionar la eliminación de datos dentro del sistema, completando así las operaciones básicas de un servicio REST.

GET	DELATE
	

Hasta este punto del desarrollo, se logró implementar un servicio web tipo **REST** funcional, el cual permite realizar operaciones básicas como consultar **(GET)**, crear **(POST)**, actualizar **(PUT)** y eliminar **(DELETE)** objetos dentro del sistema. Estas funcionalidades fueron probadas mediante Swagger, lo que permitió verificar que cada una de las peticiones responde correctamente y en el formato esperado.

El desarrollo realizado hasta ahora representa una primera fase del proyecto, enfocada en comprender el funcionamiento general de un backend, el manejo de peticiones **HTTP** y el uso de datos en formato **JSON**. Para facilitar este proceso, se utilizó una lista en memoria como almacenamiento temporal, lo que permitió validar la lógica sin necesidad de una base de datos desde el inicio.

Sin embargo, este enfoque es solo una simulación del comportamiento real de un sistema, por lo que el siguiente paso en el desarrollo consiste en integrar una base de datos relacional. Esto permitirá almacenar la información de manera persistente y trabajar con consultas reales utilizando **SQL**.

Adicionalmente, se planea mejorar la estructura del proyecto aplicando una arquitectura por capas, separando responsabilidades entre el controlador, la lógica del negocio y el acceso a datos mediante el uso del patrón **DAO**. Esto no solo hará el sistema más organizado, sino que también se ajustará a las buenas prácticas vistas en clase.

En conclusión, el trabajo realizado hasta ahora sienta las bases del sistema, permitiendo comprender su funcionamiento general. A partir de este punto, el proyecto evolucionará hacia una implementación más completa y cercana a un entorno real, incorporando base de datos, manejo de conexiones y una mejor organización del código.

Implementación de la clase ObjetoDAO

```java
package com.itm.Intercambios;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

// Esta clase se encarga de manejar la conexión con la base de datos
// y ejecutar las consultas SQL
public class ObjetoDAO {

    // Datos de conexión a la base de datos H2
    private final String URL = "jdbc:h2:mem:testdb";
    private final String USER = "sa";
    private final String PASSWORD = "";

    // Método para obtener conexión
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    // Crear tabla si no existe
    public void crearTabla() {
        String sql = "CREATE TABLE IF NOT EXISTS objetos (" +
                     "id INT PRIMARY KEY, " +
                     "nombre VARCHAR(100), " +
                     "descripcion VARCHAR(255))";

        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {

            stmt.execute(sql);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // INSERT → guardar objeto (POST)
    public void guardar(Objeto obj) {
        String sql = "INSERT INTO objetos (id, nombre, descripcion) VALUES (?, ?, ?)";

        try (Connection conn = getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, obj.getId());
            stmt.setString(2, obj.getNombre());
            stmt.setString(3, obj.getDescripcion());

            stmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // SELECT → listar objetos (GET)
    public List<Objeto> listar() {
        List<Objeto> lista = new ArrayList<>();
        String sql = "SELECT * FROM objetos";

        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Objeto obj = new Objeto(
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getString("descripcion")
                );
                lista.add(obj);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }

        return lista;
    }
}
```

En esta parte del desarrollo se creó la clase ***ObjetoDAO***, la cual se encarga de manejar la conexión con la base de datos y ejecutar las consultas SQL necesarias para el funcionamiento del sistema.

Dentro de esta clase se definieron los datos de conexión hacia la base de datos H2, así como un método para establecer la conexión utilizando ***DriverManager***, tal como se trabajó en clase.

Además, se implementaron métodos para crear la tabla de objetos en la base de datos, insertar nuevos registros mediante la instrucción SQL ***INSERT***, y consultar los datos utilizando ***SELECT***. Estos métodos permiten gestionar la información de forma más organizada y realista en comparación con el uso de estructuras en memoria.

Esta implementación representa un avance importante en el proyecto, ya que se empieza a trabajar con una base de datos real y se aplica el patrón DAO, separando la lógica de acceso a datos del resto de la aplicación.

```
<!--  H2  -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Después de integrar la base de datos al proyecto, se realizaron las pruebas necesarias para verificar que todo estuviera funcionando correctamente. Inicialmente, el sistema presentaba un error debido a la falta del driver de conexión, pero esto se solucionó agregando la dependencia de H2 en el archivo ***pom.xml***.

Una vez corregido ese problema, el servidor pudo iniciarse sin inconvenientes y se procedió a probar los endpoints desde Swagger. Al utilizar el método POST, se logró enviar información al sistema y guardarla en la base de datos. Luego, mediante el método GET, se pudo confirmar que esos mismos datos estaban siendo recuperados correctamente.

Este paso fue bastante importante, ya que se dejó de trabajar únicamente con datos en memoria y se empezó a utilizar una base de datos real, haciendo que el comportamiento del sistema sea mucho más cercano al de una aplicación real. Además, se comprobó que las consultas SQL implementadas en el DAO están funcionando correctamente y que la conexión a la base de datos se realiza sin problemas.

UPDATE + DELETE

```java
package com.itm.Intercambios;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

// Esta clase se encarga de manejar la conexión con la base de datos
// y ejecutar las consultas SQL
public class ObjetoDAO {

    // Datos de conexión a la base de datos H2
    private final String URL = "jdbc:h2:mem:testdb";
    private final String USER = "sa";
    private final String PASSWORD = "";

    // Método para obtener conexión
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    // Crear tabla si no existe
    public void crearTabla() {
        String sql = "CREATE TABLE IF NOT EXISTS objetos (" +
                     "id INT PRIMARY KEY, " +
                     "nombre VARCHAR(100), " +
                     "descripcion VARCHAR(255))";

        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {

            stmt.execute(sql);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // INSERT → guardar objeto (POST)
    public void guardar(Objeto obj) {
        String sql = "INSERT INTO objetos (id, nombre, descripcion) VALUES (?, ?, ?)";

        try (Connection conn = getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, obj.getId());
            stmt.setString(2, obj.getNombre());
            stmt.setString(3, obj.getDescripcion());

            stmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // SELECT → listar objetos (GET)
    public List<Objeto> listar() {
        List<Objeto> lista = new ArrayList<>();
        String sql = "SELECT * FROM objetos";

        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Objeto obj = new Objeto(
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getString("descripcion")
                );
                lista.add(obj);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }

        return lista;
    }

    // UPDATE → actualizar objeto (PUT)
    public void actualizar(Objeto obj) {
        String sql = "UPDATE objetos SET nombre = ?, descripcion = ? WHERE id = ?";

        try (Connection conn = getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, obj.getNombre());
            stmt.setString(2, obj.getDescripcion());
            stmt.setInt(3, obj.getId());

            stmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // DELETE → eliminar objeto
    public void eliminar(int id) {
        String sql = "DELETE FROM objetos WHERE id = ?";

        try (Connection conn = getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, id);
            stmt.executeUpdate();

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

En esta parte del desarrollo se continuó trabajando en la clase ObjetoDAO, agregando nuevas funcionalidades para completar el manejo de datos en la base de datos.
Primero, se implementó el método de actualización (UPDATE), el cual permite modificar la información de un objeto existente a partir de su id. Con este método es posible cambiar datos como el nombre o la descripción directamente en la base de datos, sin necesidad de eliminar y volver a crear el registro.
Luego, se agregó el método de eliminación (DELETE), el cual permite borrar un objeto de la base de datos utilizando su identificador. Esto hace posible gestionar de forma completa los datos, eliminando aquellos que ya no sean necesarios.
Con estos dos métodos, se completa el conjunto de operaciones básicas conocidas como CRUD (Crear, Leer, Actualizar y Eliminar), lo cual es fundamental en cualquier aplicación que maneje información.
Este avance es importante porque ahora el sistema no solo puede guardar y consultar datos, sino también modificarlos y eliminarlos directamente desde la base de datos, acercándose mucho más al funcionamiento de una aplicación real.

En esta etapa final del desarrollo, se realizó la integración entre el controlador y la capa de acceso a datos (DAO), eliminando el uso de estructuras en memoria y utilizando directamente la base de datos para gestionar la información.
Se reemplazó la lista de objetos por el uso de la clase ObjetoDAO, la cual se encarga de ejecutar las operaciones SQL correspondientes. De esta forma, los métodos GET, POST, PUT y DELETE ahora trabajan directamente sobre la base de datos, permitiendo una gestión real de los datos.
Además, se inicializó la tabla al iniciar el controlador, asegurando que la estructura necesaria exista antes de realizar cualquier operación.
Con estos cambios, el sistema deja de ser una simulación y pasa a funcionar como una aplicación real, cumpliendo con los requisitos de uso de base de datos, SQL y el patrón DAO.

```java
package com.itm.Intercambios;

import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

// Esta clase maneja las peticiones del usuario
@RestController
public class ObjetoController {

    private ObjetoDAO dao = new ObjetoDAO();

    public ObjetoController() {
        dao.crearTabla();
    }

    // GET → ver todos los objetos
    @GetMapping("/objetos")
    public List<Objeto> listarObjetos() {
        return dao.listar();
    }

    // POST → agregar un nuevo objeto
    @PostMapping("/objetos")
    public Objeto agregarObjeto(@RequestBody Objeto objeto) {
        dao.guardar(objeto);
        return objeto;
    }

    // PUT → actualizar un objeto existente
    @PutMapping("/objetos/{id}")
    public Objeto actualizarObjeto(@PathVariable int id, @RequestBody Objeto objetoActualizado) {

        objetoActualizado.setId(id);
        dao.actualizar(objetoActualizado);
        return objetoActualizado;
    }

    // DELETE → eliminar un objeto por id
    @DeleteMapping("/objetos/{id}")
    public String eliminarObjeto(@PathVariable int id) {

        dao.eliminar(id);
        return "Objeto eliminado correctamente";
```

CONCLUSIÓN:

Al finalizar este entregable, se logró construir un servicio web tipo REST completamente funcional, pasando por todas las etapas necesarias desde una implementación básica hasta una solución más cercana a un sistema real.

Al inicio, todo se trabajó con datos en memoria para poder entender cómo funcionan las peticiones HTTP y cómo responde el backend. Esto permitió enfocarse primero en la lógica de los métodos GET, POST, PUT y DELETE sin complicarse con otros factores. Luego, poco a poco, se fue mejorando la aplicación hasta integrar una base de datos real.

Uno de los avances más importantes fue la implementación del patrón DAO, ya que permitió separar la lógica de acceso a datos del resto del sistema. Gracias a esto, el código quedó más organizado y más fácil de entender, además de cumplir con la arquitectura por capas vista en clase.

También se trabajó directamente con SQL para realizar operaciones como insertar, consultar, actualizar y eliminar registros, lo cual ayudó a comprender mejor cómo se maneja la información en una base de datos relacional. A esto se suma el uso del driver de conexión y el manejo manual de conexiones, lo cual hizo el proceso más completo y cercano a lo que se haría en un entorno profesional.

Durante el desarrollo, no todo fue directo, ya que se presentaron varios errores (como problemas de conexión, dependencias o puertos), pero cada uno se fue solucionando paso a paso. Esto también fue parte clave del aprendizaje, ya que permitió entender mejor cómo funciona el entorno y cómo reaccionar ante problemas reales.

En general, el proyecto no solo cumple con los requisitos solicitados, sino que también demuestra una evolución clara en el desarrollo, pasando de algo muy básico a una aplicación estructurada, funcional y bien organizada. Además, deja una base sólida para seguir mejorando

Debido a que el desarrollo técnico se realizó inicialmente en un entorno de pruebas local para asegurar la resolución de errores críticos, el historial de Git fue consolidado en la fase final del proyecto.
Una vez validada la funcionalidad completa del CRUD y la correcta persistencia de los datos, el equipo procedió a registrar los avances en el repositorio oficial de manera secuencial. El objetivo de este procedimiento fue reconstruir la evolución lógica del software (desde el controlador básico hasta la implementación del patrón DAO) de forma limpia, organizada y libre de versiones redundantes. Por lo tanto, aunque la frecuencia de los commits sea cercana, estos reflejan fielmente las etapas de construcción detalladas en esta documentación, asegurando que el repositorio cumpla con los estándares de orden y trazabilidad requeridos para la entrega.